

Relatório do Sistema

Conciliador OFX × Planilha × Domínio — funcionalidades implementadas

1. Visão geral

Aplicativo desktop em Python/Tkinter para conciliação tripla:

- **Planilha (.xlsx)** com contas a pagar (vencimento, pagamento, valor da parcela, NF, fornecedor, CNPJ, emissão).
- **Extrato bancário (OFX)** com pagamentos efetivados — só valores negativos (saídas) são considerados.
- **Sistema Domínio (Escrita Fiscal)** via ODBC (read-only) — puxa parcelas da tabela *bethadba.efentradaspar* com JOIN em *efentradas* e *effornece*, com status calculado a partir de *efentradaspag* (Aberto / Parcial / Paga).

Fluxo principal: carregar as 3 fontes → casar Planilha × OFX (match exato + sugestões com tolerância) → cruzar o resultado com o Domínio por data de vencimento + valor + número da NF (segunda fase).

2. Leitura de dados

Planilha Excel (.xlsx) — parser_xlsx.py

- **Deteção automática da linha do cabeçalho** (procura nas primeiras 15 linhas, pula linhas em branco, títulos, logos).
- **Auto-deteção dos 7 campos obrigatórios por nome**, com dezenas de aliases: *Data vencimento*, *Data pagamento* (data pgto / pago em / data baixa / data quitação / data liquidação / data compensação...), *Data emissão*, *Valor*, *Nº NF*, *CNPJ*, *Fornecedor*.
- **Fallback por conteúdo**: quando o nome não bate, classifica a coluna pelo tipo das células — mais datas → Data, mais números → Valor, textos mais longos → Fornecedor.
- **Diálogo de mapeamento manual** com preview ao vivo das primeiras 10 linhas; linhas em vermelho destacam quando alguma data ou valor não pôde ser convertido.
- **Conversão robusta de valores** no formato brasileiro: R\$ 1.234,56, vírgula decimal, parênteses para negativo.
- **Múltiplos formatos de data**: dd/mm/aaaa, aaaa-mm-dd, dd-mm-aaaa, dd/mm/aa, dd.mm.aaaa e células nativas de data do Excel.

Extrato OFX — parser_ofx.py

- Leitura via *ofxparse*.
- **Filtra automaticamente apenas pagamentos** (valores negativos), ignorando recebimentos — o app mostra a contagem de ignorados.
- Inverte o sinal para casar com a planilha (que tem valores positivos).

Sistema Domínio (ODBC) — parser_dominio.py

- **Padrão Janco**: credenciais em *data/dominio_config.json* (gitignored), separadas do *config.json* de preferências.
- **connect_dominio(readonly=True)** como context manager para scripts; **open_connection()** para conexões longas mantidas pela GUI.
- **load_odbc_config / save_odbc_config** para persistir e ler credenciais.
- **listar_empresas(conn)** lê *bethadba.geempre* com fallback entre variantes de nomes de coluna (RAZA_EMP / RAZAO_EMP / NOME_EMP).

- **listar_tabelas** filtra schema *bethadba* por padrão; checkbox no diálogo libera ver todos os schemas.
- **amostra** usa fallback TOP / FIRST / LIMIT para compatibilidade entre dialetos do SQL Anywhere.
- **extrair_pagamentos** aceita dois modos (tabela+WHERE ou SQL livre) e injeta automaticamente *CODI_EMP = ?* como parâmetro pyodbc. Detecta colunas opcionais *status_parcela* e *valor_pago* vindas do SQL e armazena em *Transacao.extras*.
- **Gotchas documentados:** DISTINCT + ORDER BY exige aliases do SELECT (não nomes qualificados — erro -854 no SA).

3. Modelo de dados

Campos obrigatórios da planilha (7):

- **Data vencimento** — usada no match com Domínio (vcto_entp)
- **Data pagamento** — usada no match com OFX (data de compensação)
- **Data emissão** — data do documento fiscal
- **Valor da parcela**
- **Número da NF**
- **CNPJ do fornecedor**
- **Nome do fornecedor**

Campos obrigatórios do Domínio (6 — sem data_pagamento):

- Data vencimento (vcto_entp), Valor, Data emissão, Nº NF, CNPJ, Fornecedor

O **OFX** é diferente: tem apenas *data de pagamento*, *valor* e *memo* (do banco). O memo aparece nas abas onde o OFX está envolvido.

Query padrão usada no Domínio

JOIN das 3 tabelas + sub-query agregando pagamentos para derivar status (Aberto/Parcial/Paga), parametrizado por empresa:

```
SELECT par.vcto_entp AS data_vencimento, par.vlor_entp AS valor_parcela, par.parc_entp AS
numero_parcela, ent.ddoc_ent AS data_emissao, ent.ume_ent AS numero_nf, forn.cgce_for AS
cnpj_fornecedor, forn.nome_for AS nome_fornecedor, COALESCE(pag.total_pago, 0) AS
valor_pago, CASE WHEN COALESCE(pag.total_pago,0)=0 THEN 'Aberto' WHEN
COALESCE(pag.total_pago,0)>=par.vlor_entp THEN 'Paga' ELSE 'Parcial' END AS status_parcela
FROM bethadba.efentradaspar par JOIN bethadba.efentradas ent ON ent.codi_emp=par.codi_emp
AND ent.codi_ent=par.codi_ent JOIN bethadba.efornece forn ON forn.codi_emp=ent.codi_emp
AND forn.codi_for=ent.codi_for LEFT JOIN (SELECT
codi_emp,codi_ent,parc_entp,SUM(vpag_entp) AS total_pago FROM bethadba.efentradaspag WHERE
pgto_entp IS NOT NULL GROUP BY codi_emp,codi_ent,parc_entp) pag ON
pag.codi_emp=par.codi_emp AND pag.codi_ent=par.codi_ent AND pag.parc_entp=par.parc_entp
WHERE par.codi_emp=?
```

4. Lógica de conciliação

Fase 1 — Planilha × OFX (matcher.py)

- **conciliar_automatico**: match exato por (data_pagamento, valor) quantizado para 2 casas. Da planilha usa *data_pagamento* (quando mapeada — recomendado), com fallback para *t.data* (vencimento) se não houver.
- **gerar_sugestoes**: pares de pendentes onde a diferença $|\Delta dias| \leq 2$ e $|\Delta valor| \leq R\$ 10,00$. Ordenado pelos mais próximos.
- **diferenca(p, o)**: calcula $\Delta dias$ e $\Delta valor$ entre planilha e OFX (usando *data_pagamento*).
- **Conciliação manual**: selecionar 1 em cada lado dos Pendentes e clicar "Conciliar selecionadas →".

Fase 2 — Pares × Domínio (main.py)

- **_filtrar_conciliados_por_dominio**: para cada par P × OFX, busca uma transação do Domínio que case por (*data_vencimento* + *valor* + *NF*). Quando encontra, marca *par.dominio*. Cada transação do Domínio só pode casar com 1 par.
- Disparado automaticamente após: Conciliar, Carregar pagamentos do Domínio (se já havia conciliação), aceitar sugestão, conciliação manual, e botão Comparar com Domínio.
- Aba **Conciliados × Domínio** mostra apenas pares onde *par.dominio* está preenchido.
- Aba **Comparação** mostra a visão tripla (verde / amarelo / vermelho) incluindo pagamentos do Domínio sem conciliação.

5. Interface gráfica (Tkinter) — main.py

Barra de ações (4 linhas no topo)

- **Linha 1 — Planilha**: Abrir planilha (.xlsx) | Editar colunas | label
- **Linha 2 — OFX**: Importar OFX | label
- **Linha 3 — Domínio**: Conectar Domínio | Selecionar empresa | Configurar fonte | Carregar pagamentos | label de status
- **Linha 4 — Ações**: Conciliar | Comparar com Domínio | resumo

Diálogo de mapeamento da planilha — DialogoMapeamento

- 7 dropdowns (todos obrigatórios) pré-selecionados com a sugestão automática.
- **Preview ao vivo** mostra as primeiras 10 linhas com as 7 colunas mapeadas — atualiza a cada mudança de dropdown.
- **Linhas em vermelho** quando vencimento, pagamento, emissão ou valor não puderam ser convertidos.
- Status com contagem N/M linhas válidas.

Diálogos do Domínio — dialogos_dominio.py

- **DialogoConexao**: DSN + usuário + senha; testa a conexão (read-only) e persiste em *data/dominio_config.json*.
- **DialogoSelecionarEmpresa**: lista buscável de *bethadba.geempre* (código + razão social + CNPJ); duplo-clique seleciona.
- **DialogoFonte**: alterna entre *Tabela + WHERE* e *Query SQL manual*; lista tabelas do schema *bethadba* (com opção de ver outros schemas); preview até 100 linhas; mapeia os 6 campos.
- No modo SQL, quando há ? na query, o preview e a extração passam automaticamente o CODI_EMP da empresa selecionada como parâmetro pyodbc.

Filtros das abas de dados

- **Busca global:** campo "Buscar:" filtra linhas que contenham o texto em qualquer coluna visível (case-insensitive, em tempo real).
- **Status (só Domínio dados):** dropdown "Todos / Aberto / Parcial / Paga" filtra pelo status calculado da parcela.
- **Filtros estilo Excel por coluna:** clica no cabeçalho de qualquer coluna → abre popup com checkboxes dos valores únicos daquela coluna. Tem busca interna, marcar/desmarcar tudo, OK/Cancelar. Cabeçalho mostra "□" sem filtro e "▼ □" com filtro ativo.
- Todos os filtros combinam com **AND**. Botão "Limpar" zera tudo de uma vez (global + por coluna + status).

Abas do Notebook (8 abas em 2 blocos)

#	Aba	Conteúdo	Ações / Notas
0	Planilha	Dados crus da planilha (linha original, vencimento, pagamento, emissão, valor, NF, CNPJ, fornecedor).	Filtros Excel-style por coluna
1	OFX	Pagamentos do extrato bancário (data de compensação, valor, memo do banco).	Filtros Excel-style por coluna
2	Domínio dados	Parcelas vindas do Domínio (vencimento, emissão, valor parcela, valor pago, status, NF, CNPJ, fornecedor).	Filtros + dropdown Status
3	Conciliados	TODOS os pares Planilha × OFX (verde = auto, azul = manual). Mostra Δ dias / Δ valor.	Desfazer conciliação manual
4	Pendentes	Dois lados: "Só na planilha" (com NF e fornecedor) e "Só no OFX" (com memo).	Conciliar selecionadas →
5	Sugestões	Pares aproximados P × OFX ($\Delta \leq 2d$ e $\Delta \leq R\$ 10$).	Aceitar como conciliação
6	Conciliados × Domínio	APENAS pares triple-matched: P × OFX que também batem no Domínio (vencimento + valor + NF). Coluna Status colorida (Aberto/Parcial/Paga).	—
7	Comparação	Visão tripla: □ em tudo, □ falta na contabilidade, □ só no Domínio (sem conciliação).	—

Comportamentos transversais

- Resumo no topo com contagem por categoria; títulos das abas mostram número entre parênteses (ex.: *Conciliados (142)*).
- Carregar nova planilha/OFX **limpa automaticamente** os resultados anteriores de conciliação.
- Aviso quando o mapeamento produz **0 lançamentos**.
- Aviso quando **Carregar pagamentos** sem empresa selecionada — evita puxar dados de todas as empresas misturadas.
- Conciliação **manual** entre pendentes; **aceitar sugestão**; **desfazer conciliação manual** devolve o par para pendentes e recalcula sugestões.

6. Persistência e configuração

- **config.json** (raiz do projeto, gitignored): preferências do app — mapeamento de colunas da planilha, empresa selecionada do Domínio, fonte do Domínio (modo, tabela ou SQL, WHERE, mapeamento das 6 colunas).
- **data/dominio_config.json** (gitignored): credenciais ODBC — DSN, usuário, senha em texto.
- **Migração automática**: configs antigas com `cfg["dominio"]` misturado são divididas automaticamente na primeira execução do app atualizado.

7. Infraestrutura

- **requirements.txt**: openpyxl, ofxparse, pyodbc, reportlab.
- **.gitignore**: protege `config.json`, `data/dominio_config.json`, `.venv/`, `__pycache__`/, planilhas, OFX e CSV.
- **README.md**: guia completo de instalação, configuração do DSN ODBC para o Domínio (SQL Anywhere 17), fluxo de uso, convenções do projeto Janco.
- **iniciar.bat**: clique-duplo no Explorer → faz `git pull` + ativa venv + abre o app, sem precisar abrir terminal.
- **gerar_pdf.py**: script para regenerar este relatório.
- **Repositório**: github.com/Fernandovini2607/conciliador (privado).

8. Convenções herdadas do projeto Janco

- Toda tabela do Domínio Escrita Fiscal vive no schema **bethadba** (sempre prefixar).
- Conexão ODBC é sempre **read-only** (o usuário ODBC do Domínio não tem GRANT de UPDATE; qualquer escrita exige RPA — fora do escopo deste app).
- **Gotcha SQL Anywhere**: DISTINCT + ORDER BY exige usar os aliases do SELECT, não nomes qualificados (erro -854 com `p.CODI_GRU`; use `grupo` se aliasou).
- Empresas vivem na mesma base; separação é por **CODI_EMP** em cada tabela. O app injeta esse filtro automaticamente quando há ? no SQL.

9. Tabelas do Domínio utilizadas

- **bethadba.efentradas** — cabeçalho de NF de entrada (`codi_emp`, `codi_ent`, `codi_for`, `nume_ent`, `ddoc_ent`, `vcon_ent`).
- **bethadba.efentradaspar** — parcelas/duplicatas geradas no import do XML (`vcto_entp`, `vlor_entp`, `parc_entp`).
- **bethadba.efentradaspag** — pagamentos registrados (`pgto_entp`, `vpag_entp`). Usada para calcular o status agregado.
- **bethadba.effornece** — cadastro de fornecedores (`codi_for`, `nome_for`, `cgce_for`).
- **bethadba.geempre** — cadastro de empresas (`codi_emp` + razão + CNPJ). Usada para a lista da seleção de empresa.

10. O que ainda não existe

- Tolerância configurável via UI (hoje fixa em 2 dias / R\$ 10,00).
- Exportação do resultado para Excel/PDF/CSV (com cores).
- Tratamento de planilhas com colunas separadas de Crédito/Débito.
- Match por similaridade textual (descrição ou fornecedor).
- Suporte a múltiplas contas no mesmo OFX.

- Histórico de conciliações entre execuções.
- Conciliação manual diretamente na aba do Domínio.
- Escrita de volta no Domínio (exigiria RPA via pywinauto).